

Python 后端求职作品集深度规划报告

核心判断

先说结论，再展开。

基于你给出的两个仓库快照，我的判断很明确：**JobPilot 更接近“可以先打磨到简历安全线”的项目，VoiceMate 更接近“概念和边界已经清楚，但真正价值还没被代码证明”的项目。**前者已经有比较明显的传统后端工程骨架：应用入口、数据库迁移、认证链路、岗位导入/规范化/查询、Redis/Celery/Compose 资源骨架、测试分层。后者目前更像一套**方向正确的 AI 应用后端骨架**：`backend service` / `ai service` 的边界有了，`ReadingSession` / `ReadingTurn` / `ReadingReview` 的对象有了，`TutorPolicy` 的想法也清楚，但大量核心链路仍处于占位或 stub 阶段。

这也是为什么这两个项目天然适合做成“一稳一亮”的组合：JobPilot 负责证明你能做通用 Python 后端，VoiceMate 负责证明你能把 AI 能力落到具体产品场景。但在当前阶段，它们的成熟度并不对等，所以绝不能平均发力。FastAPI 的自动 OpenAPI 文档、依赖注入和 API 导向开发方式，很适合你这种需要快速做出可解释 API 闭环的作品集；SQLAlchemy 的 `AsyncSession` 在 2.x 风格下可以完整支持 ORM，但官方也强调需要显式管理事务边界、避免异步场景下的隐式 I/O；PostgreSQL 已经内置全文检索、GIN/GiST 索引和 `pg_trgm` 相似搜索，Redis 原生支持 TTL，Celery 也原生支持任务重试和状态跟踪，这些都说明 **JobPilot 完全可以在不引入过多新基建的前提下，做出足够像样的工程闭环。** ¹

VoiceMate 这边，官方资料给出的方向也很明确：OpenAI 把**低延迟 live audio** 和有边界的 **request-based audio** 明确区分，建议先决定是做 **speech-to-speech** 还是 **显式 STT→推理→TTS 的链式 pipeline**；LiveKit Agents 则明确是把你的程序接成房间里的实时 participant，而 token 需要由后端生成。这意味着你现在最合理的路线不是“一口气做全实时多模态系统”，而是**先做 pipeline 模式下的多模态陪读闭环，再把 realtime 当进阶阶段**。这既符合官方推荐的架构分层方式，也更符合一个月、AI 辅助开发且以求职为目标的现实约束。

2

维度	JobPilot	VoiceMate	我的判断
作品集类型	传统 Python 后端业务系统	AI 应用后端 / 实时智能体后端	两者互补
当前成熟度	已有工程骨架，能继续收口	方向清楚，但核心实现仍偏占位	先打磨 JobPilot
对普通 Python 后端岗的价值	很高	中高	JobPilot 更稳
对 AI 应用后端岗的价值	中高	很高	VoiceMate 更亮
被面试官快速理解的难度	低	中高	JobPilot 更容易被认可
被质疑“AI 生成痕迹重”的风险	中	高	VoiceMate 风险更大

维度	JobPilot	VoiceMate	我的判断
简历排序建议	一般放第一	一般放第二	除非 JD 明确偏 AI 应用后端
首月投入优先级	最高	次高	先主后辅

项目定位与取舍

如果把这两个项目包装成作品集，我建议你不要把它们写成两个“都会一点”的项目，而要写成两个**角色完全不同的项目**。

JobPilot 的推荐定位是：

“面向求职场景的 **Python 后端业务系统**”，核心卖点不是“抓岗位”也不是“做 AI”，而是**数据建模、鉴权、查询、去重、缓存、异步任务、测试和工程化**。它应该像一个缩小版但真实的业务后端，而不是一个大杂烩练手仓库。

VoiceMate 的推荐定位是：

“面向儿童绘本陪读场景的 **AI 应用后端与实时会话架构项目**”，核心卖点不是“接了大模型”，而是**如何组织多模态上下文、如何设计 TutorPolicy、如何保存会话状态、如何做 fallback、如何把实时链路拆成可解释的边界**。它不是通用 Agent 平台，也不是内容平台，更不是完整教育系统。

这两个项目是互补的，而且互补得非常好。一个证明你能做**稳态的、规则明确的、结构化数据驱动的后端系统**；另一个证明你不只会 CRUD，也能处理**模型调用、状态机、实时互动、链路收敛和产品化边界**。这组组合比“两个传统 CRUD 项目”更有辨识度，也比“两个 AI 套壳项目”更可信。FastAPI 的 API 导向能力、Django 的 batteries-included 思路、Flask 的轻量扩展哲学、PostgreSQL 的内置全文检索、Docker Compose 的多容器编排，以及 pytest 的 fixture/`conftest.py` 共享上下文，都指向同一件事：**对于求职作品集，最重要的是能讲清楚系统边界与工程取舍，而不是同时堆很多框架名。** ³

直接回答你最关心的选择题：

问题	结论
JobPilot 和 VoiceMate 分别适合包装成什么类型的作品集项目	JobPilot 是经典后端业务系统；VoiceMate 是 AI 应用后端与实时会话架构项目
两个项目是否互补	是，且互补度高
哪个更适合作为主项目	JobPilot
哪个更适合作为辅助项目	VoiceMate
如果时间有限优先投入哪个	先 JobPilot，至少做到“简历安全线”；然后再做 VoiceMate 的闭环
哪个更稳，哪个更亮	JobPilot 更稳；VoiceMate 更亮
哪个更容易被面试官认可	JobPilot 更容易被广泛认可
哪个更容易被质疑 AI 痕迹重 / 不真实 / 过度设计	VoiceMate 更容易；JobPilot 也会因为“未来模块太多”而被怀疑

问题	结论
它们适合写在简历什么位置	普通后端岗：JobPilot 第一、VoiceMate 第二；AI 应用后端岗：VoiceMate 可前置，但前提是已经有真实闭环，不是纯概念图

我再补一句更直接的话：如果首月只能把一个项目做到让人放心，就把 JobPilot 做到扎实；如果首月把两个项目都做成“能跑但讲不清”的样子，效果会比只有一个扎实项目更差。

阶段规划

你的阶段规划不应该按“功能越多越好”来排，而应该按三条线来排：

- **跑通线**：最小可运行闭环。
- **简历线**：已经足够写进简历，并且能被追问。
- **加分线**：能应对更深入的面试追问，但不是首月必需。

为什么要这么分？因为 JobPilot 这类后端系统，真正的价值来自数据模型、索引、缓存、幂等和测试，而不是功能数量；VoiceMate 这类 AI 应用后端，真正的价值来自会话对象、策略层、延迟/fallback、服务边界，而不是堆 Realtime、RAG、LangGraph、LiveKit、向量库这些名词。PostgreSQL 已经给了你足够强的检索底座；Elastic 自身的 Query DSL 和映射系统很强，但也意味着更高的接入复杂度，甚至官方都不推荐直接把严格的 query_string 当搜索框入口。OpenAI 官方也明确把语音链路先分成“live audio”与“chained pipeline”两种路线，所以 VoiceMate 必须先做 pipeline，再考虑 realtime。 4

规划项	JobPilot	VoiceMate
MVP 阶段	seed/raw import -> normalize -> fingerprint 去重 -> jobs list/detail/filter -> register/login/me -> 收藏或目标岗位。注意：MVP 至少要有有一个“用户动作”，否则只是公开岗位接口。	books/pages -> create session -> submit one pipeline turn -> TutorPolicy 选动作 -> ResponseComposer 生成回复 -> 保存 turn -> 生成 review。注意：MVP 不要求先做 LiveKit。
作品集阶段	在 MVP 上补齐 job_collections、job_targets、user_skills、job match、Redis 缓存、核心测试、迁移说明、Docker Compose 本地跑通。做到这一步，才适合放简历。	在 MVP 上补齐 ReadingSession / ReadingTurn / ReadingReview 持久化、可解释的 TutorPolicy、fallback、latency 记录、backend/ai service 责任边界、基础安全约束。做到这一步，才适合放简历。
面试加分阶段	Celery 异步摄入、失败重试、任务状态；PostgreSQL FTS/pg_trgm 或再往后接 ES；热门技能统计；Cache Aside 失效策略。	LiveKit token/room info 接口；realtime 模式接通；多模型 provider fallback；更细的 turn telemetry；儿童场景安全收敛。
必须先做	数据模型、认证链路、岗位导入/去重、搜索筛选、用户隔离、测试。	会话模型、turn 记录、TutorPolicy、ResponseComposer、pipeline turn 接口、fallback。
应该暂时砍掉	study_tasks、knowledge、questions、RAG、AI 简历分析、Kafka、复杂 RBAC、复杂后台。	通用 Agent 平台、内容平台化、完整儿童教育系统、复杂后台、通用 RAG 平台、多 agent 协作、长期记忆。
过度设计警报	一开始就上 ES、RabbitMQ+Celery+Kafka 三件套、第三方登录全家桶、微服务拆分、Prometheus/Grafana 全套但没指标。	一开始就上 LiveKit + OpenAI Realtime + 本地模型 + LangGraph + RAG + 向量库；或者先写一堆“Agent 名词”但没有 turn persistence。

规划项	JobPilot	VoiceMate
最容易做成玩具 Demo 的地方	只有静态 seed 数据、没有 raw/normalized 分层、没有唯一约束、没有用户归属校验、没有边界错误测试。	只有“创建 session + 返回 stub 文本”、没有真正的 TeachingAction、没有 turn 保存、没有 latency/fallback、没有 session review。
如何更接近真实工程	Alembic 迁移历史、唯一约束与索引、失败表/错误记录、owner-scope 授权、缓存 miss/hit 路径、端到端测试、配置区分 dev/test。	明确 backend 与 ai service 的接口边界；把 policy_action / rationale / signals / latency_ms / fallback_used 真正落进 turn 记录；把“非目标”写进 README；对外只暴露少量稳定 API。

还要再点两个当前仓库最现实的问题。

第一，JobPilot 的未来模块已经列得太多。study_tasks / knowledge / questions 这些模块作为远期方向可以留在规划里，但如果首月没有真实代码，不要在 README 首页把它们当成项目主线，更不要写进简历。空目录和空 README 不会加分，只会让人怀疑是先搭了 AI 生成的架子。

第二，VoiceMate 当前的概念密度高于实现密度。

ReadingCoachAgent / TutorPolicy / ResponseComposer / SessionMemory / pipeline / realtime / RAG / fallback 这些名词都对，但它们必须尽快压缩成一个真实的 turn 闭环，否则面试官会默认这是“有设计文档、没落地”。

AI 辅助开发下的掌握策略

你现在最需要建立的，不是“怎么让 AI 多写代码”，而是哪些东西绝不能外包给 AI。

我建议你把自己在项目里的职责定义成五件事：

需求边界、数据/状态模型、事务与失败语义、验收标准、代码审查。

AI 可以加速脚手架、CRUD、样板测试、Dockerfile、基础 schema，但不能替你定义不变量。SQLAlchemy 2.x 把事务边界显式化，AsyncSession 也明确要求你避免隐式 I/O；FastAPI 的依赖注入和安全依赖可以复用，但前提是你知道自己在复用什么；OpenAI 的 voice agents 指南也明确强调要先确定音频架构，再设计 agent 工作流。换句话说：你不能把“系统怎么工作”交给 AI，只能把“重复的写法”交给 AI。 ⁵

你应当把“必须自己吃透”的模块，压缩到下面这张表里。这里的“吃透”不是背源码，而是至少能画图、能口述、能改一两个边界条件。

模块	JobPilot 你必须吃透什么	VoiceMate 你必须吃透什么	最低要求
数据模型	users / profiles / auth_identities / password_credentials / job_posts / job_details / job_sources 的拆分理由，唯一约束、外键、索引	ReadingSession / ReadingTurn / ReadingReview 为什么是三个对象，而不是一坨 JSON	能画出 ERD 或对象关系图

模块	JobPilot 你必须吃透什么	VoiceMate 你必须吃透什么	最低要求
事务与一致性	注册登录、refresh token 轮换、raw import -> normalized upsert 的事务边界	session 创建、turn 保存、review 生成的时机；哪些是同步，哪些可以异步	能说清 rollback 点和幂等点
核心业务算法	fingerprint 去重、搜索筛选条件、用户收藏/目标岗位唯一约束、用户技能 match	TutorPolicy 如何从输入信号选择 explain/ask/hint/correct/follow/wrap	能手写伪代码
安全与权限	JWT access/refresh、logout、owner-scope 查询、越权测试	session access、room token 下发、基础防滥用/儿童安全收敛	能说出威胁点和最小防护
性能与可靠性	哪些字段建索引，缓存哪些接口，Celery 重试和去重怎么做	latency budget、fallback 触发条件、慢路径和快路径怎么分流	能回答“压力变大怎么办”
可测试性	单测测什么、API 测什么、集成测什么	policy 单测、turn 闭环测试、fallback 测试、review 测试	能列出 5 个关键测试用例

你问“哪些模块必须自己理解、review，甚至能半手写”，我给你更直接的答案：

- **JobPilot 里必须半手写或深度改写的部分：**

normalize_job_draft、fingerprint 生成逻辑、搜索 query builder、refresh token 轮换/消费、owner-scope 权限校验、核心迁移脚本。
这些地方如果完全是 AI 生成且你讲不清，项目就不属于你。

- **VoiceMate 里必须自己理解并最好能画时序图的部分：**

backend service 与 ai service 的边界、create session -> one turn -> save turn -> review 的时序、TutorPolicy 的输入/输出、ResponseComposer 的职责、fallback 逻辑、是否需要 realtime transport。
这些地方如果只是“类名很好听”，面试一追问就会露馅。

- **可以让 AI 快速写，但你必须解释设计思路的部分：**

基础 router、Pydantic schema、普通 repository、测试脚手架、Docker/Compose、CI YAML、简单 CRUD、stub adapter。

但你至少要解释：**为什么是这些层、为什么这样命名、为什么这些字段存在、为什么这个错误码这样返回。**

针对“每次 AI 生成代码后该怎么审查”，我建议你固定用下面这张审查表，不管项目大小都照着走。

维度	你必须检查什么	典型红旗
业务逻辑	输入是否合法、状态变化是否符合业务、是否漏掉边界条件	happy path 能跑，异常路径全空着
数据模型	字段是否过多/过少、是否允许错误的 <code>NULL</code> 、唯一约束是否正确、关系是否过深	一个表塞所有字段；或者为了“优雅”过度拆表
API 设计	URL 是否表达资源、状态码是否合理、分页/筛选是否一致	一个接口包办所有动作，或者命名很炫但不 REST
异常处理	业务异常、鉴权异常、第三方调用异常、重试异常是否区分	<code>except Exception: pass</code> ；或统一返回 200
安全问题	密码哈希、token 生命周期、敏感字段输出、越权访问、输入校验	直接信任前端传入 <code>user_id</code> ；日志里打印 token/密钥
测试覆盖	是否测了失败用例、重复提交、权限边界、缓存失效、fallback	只有接口成功用例，没有异常用例
可维护性	命名是否稳定、函数是否过长、是否有重复逻辑、依赖是否反向污染	service/repository/router 分层齐全，但每层只转发一遍
是否过度设计	当前场景真的需要 MQ / ES / 向量库 / 多 agent 吗	先堆技术，再找场景

面试时关于 AI 的表述，建议你既诚实，也不要把自己讲成“只是按运行键的人”。我建议你准备下面这种说法：

我会用 AI 提高编码速度，但没有把设计权交给 AI。我主要负责需求拆解、接口契约、数据模型、关键路径设计、失败用例、代码 review 和测试补齐。AI 主要帮我生成脚手架、重复代码和测试初稿；像鉴权、去重、会话状态、策略层、fallback 这些核心模块，我都做了手动修改并能解释取舍。

如果被直接问“这个项目是不是 AI 写的”，可以更直接一些：

是 AI 辅助开发，不是 AI 自主完成。我不会回避这一点。但我会把 AI 生成的代码当草稿，而不是当结论。项目里哪些表要拆、哪些接口要保留、哪些异常要显式处理、哪些测试必须补，是我自己定的；关键模块我也能现场画图说明或改代码。

你需要准备的“我真的理解项目”的证据，不是嘴上说说，而是材料：

- 一张 **ER 图** 或对象关系图；
- 一张 **关键时序图**；
- 两三段你亲自改过的关键代码 diff；
- 五个你认为最重要的测试用例及其理由；
- 一页“**为何不用 X，而选 Y**”的设计取舍说明；
- 一个可以本地跑通的 demo 脚本；

- 每个项目一份 “Implemented / Planned / Dropped” 状态表。

这些证据，比 “我熟悉某某框架” 有用得多。

技术栈取舍

从求职价值看，你现在应该追求的是 “**少而真**”，不是 “全都沾一点”。FastAPI 的 API、OpenAPI 和 DI 能力非常适合作为两个项目的一致底座；Django 天然带 auth、permissions、cookie-based sessions 和 admin，适合你补框架视野，但不值得在这两个项目里再重写一遍；Flask 是轻量微框架，本身没问题，但对于你现在的目标，它不能明显增加作品集含金量。SQLAlchemy 的异步 ORM、PostgreSQL 的 FTS/GIN/`pg_trgm`、Redis 的 TTL、Docker Compose 的多容器编排、pytest 的 fixture 机制、GitHub Actions 的 CI 工作流，这一整套已经足够支撑 “传统后端作品集” 的高质量落地。⁶

在 AI 应用后端这边，你要接受一件事：**Realtime、LiveKit、本地模型、RAG、LangGraph 这些不是一个层级的东西**。OpenAI 官方明确建议先决定音频架构，再决定 agent 工作流；Responses API 天然支持文本和图片输入，适合 pipeline 闭环；Realtime 更适合低延迟 live audio；LiveKit 负责房间和 transport，而 token 必须后端生成。也就是说，你在首月里完全可以只做：**多模态上下文 + TutorPolicy + turn persistence + fallback**，而不必同时背上所有 “AI 基础设施”。⁷

技术项	结论	取舍建议
FastAPI	必须做	两个项目统一底座，继续沿用即可
Django	了解即可	补 auth/admin/ORM 视野；除非 JD 明确偏 Django，否则不重写项目
Flask	了解即可	理解轻量框架思路即可，不要为它再开第三个项目
SQLAlchemy	必须做	你必须真正掌握 session、transaction、relationship loading、async pitfalls
Django ORM	了解即可	作为对比视角，不作为当前项目主战场
PostgreSQL	必须做	比 MySQL 更适合 JobPilot 的搜索与后续扩展
MySQL	了解即可	会基本概念即可，当前作品集优先 Postgres
Redis	建议做	JobPilot 用于缓存/token；VoiceMate 可用于短期会话态或 rate limit
RabbitMQ / Celery / Kafka	Celery 建议做；RabbitMQ 加分；Kafka 暂不建议做	Celery 先承接异步导入；RabbitMQ 只在你要讲 ack/可靠投递时再上；Kafka 首月没必要
Elasticsearch	面试加分	JobPilot 首版先用 PostgreSQL FTS/ <code>pg_trgm</code> ，别首月先上 ES
WebSocket / LiveKit / Realtime	VoiceMate 中 WebSocket 了解、LiveKit/Realtime 进阶加分	先 pipeline，后 realtime；同时只选一个实时 transport 路径

技术项	结论	取舍建议
OpenAI API / 本地模型 / 多模态模型	OpenAI API 建议做；本地模型了解即可；多模态能力建议做但只选一个 provider	首月优先可验证性，不优先本地部署
RAG	了解即可	VoiceMate 只做非常局部的检索，不做通用 RAG 平台
Docker / Docker Compose	必须做	作品集的启动体验非常重要
pytest	必须做	两个项目都需要关键路径测试
CI/CD	建议做	最小化 GitHub Actions: lint + test 即可
日志、监控、健康检查	基础必须做，完整监控加分	health + structured log 必做；Prometheus/Grafana 只有在有真实指标时再强调
限流、鉴权、权限模型	鉴权必须做；限流建议做；复杂权限模型暂不建议	JobPilot 先做 owner-scope；VoiceMate 先做 session/token 边界
OpenAPI / Swagger	必须做	FastAPI 自动文档天然就是你的展示面
前端页面或纯后端 API	纯后端 API 足够	JobPilot 不需要前端；VoiceMate 可选一个极简 demo 页面，但不要扩成前端项目

关于消息系统，我建议你只记住一句：**Celery 是任务执行层，不等于消息队列**。Celery 自己支持多种 broker/result backend，任务可以 retry、可以跟踪状态；RabbitMQ 的价值在于工作队列、ack、prefetch、durability 等可靠投递语义。对 JobPilot 来说，“**Celery + Redis**”比“**RabbitMQ + Celery + Kafka 一起上**”更合理。⁸

关于监控，我的建议也很克制：**别先炫工具，先做指标**。Prometheus 的价值在于暴露和采集指标，Grafana 的价值在于可视化与告警。你真正该做的是：JobPilot 导出 request latency / cache hit / ingestion failure；VoiceMate 导出 turn latency / fallback count / review generation time。没有这些指标，Prometheus/Grafana 只是摆设。⁹

亮点设计与面试风险

你要写进简历的亮点，必须满足五个条件：**具体、可验证、能追问、和代码有对应关系、不过度承诺**。下面我按你要的格式直接给。

项目一：JobPilot

- **推荐定位：**
面向求职场景的 Python 后端业务系统，重点展示认证鉴权、数据建模、岗位导入清洗去重、复杂查询、缓存、异步任务和测试工程化。
- **核心亮点 1：岗位数据导入、清洗与指纹去重链路**

- 做了什么：设计 `raw_job_records -> normalized_job_posts/job_details -> fingerprint 唯一约束` 的导入链路。
- 为什么这样设计：招聘源格式不稳定、重复率高，先保留 raw，再规范化，便于重放、排错和幂等导入。
- 解决了什么问题：避免同一岗位重复入库，搜索层始终面向结构化数据。
- 体现什么后端能力：数据建模、事务、唯一约束、幂等、失败处理。
- 面试时怎么展开讲：从“重复消息怎么处理”“唯一冲突怎么更新”“fingerprint 怎么选字段”往下讲。
- **核心亮点 2：面向筛选场景的数据建模与查询设计**
 - 做了什么：拆分 `job_posts / job_details / job_sources`，把热查询字段和冷字段分离，围绕关键词、城市、薪资、工作方式、经验等级做组合筛选。
 - 为什么这样设计：岗位列表和岗位详情的读取模式不同，不应该所有字段一把查。
 - 解决了什么问题：提升查询可解释性和后续索引优化空间。
 - 体现什么后端能力：关系建模、ORM 查询、索引设计、分页与查询性能意识。
 - 面试时怎么展开讲：从“为什么不用一个大表”“什么时候该接 PostgreSQL FTS/ES”“为什么这个字段要入索引”展开。
- **核心亮点 3：认证与用户侧工作台的边界设计**
 - 做了什么：拆分 `users / auth_identities / auth_password_credentials`，实现注册、登录、refresh token、logout，并围绕收藏/目标岗位/技能画像做 owner-scope 隔离。
 - 为什么这样设计：用户主体、登录身份、密码凭证不是一回事，拆开以后更容易扩展第三方登录、更清晰地做权限边界。
 - 解决了什么问题：避免把认证、资料、业务状态全部搅在一张表里。
 - 体现什么后端能力：鉴权、授权、用户隔离、接口契约、越权测试。
 - 面试时怎么展开讲：从“为什么不用一个 users 表全存”“refresh token 怎么失效”“收藏接口如何防越权”展开。
- **可选加分亮点：**

Cache Aside、filter-options 缓存、Celery 异步摄入与失败重试、热门技能统计、导入错误记录与任务状态查询。
- **不建议强行包装的点：**

八股题推荐、学习任务系统、知识库、RAG、AI 简历诊断、复杂 RBAC、Prometheus/Grafana 全家桶。没做实，就别提。
- **简历写法初稿：**
 - 设计并实现岗位数据导入链路：对外部岗位原始记录进行规范化入库，基于 fingerprint 与唯一约束实现幂等去重，并保留 raw/normalized 分层以支持排错与重放。
 - 基于 FastAPI + SQLAlchemy 实现岗位列表、详情与多条件筛选 API，围绕关键词、城市、薪资、经验等条件组织查询，并为高频筛选结果引入 Redis 缓存。
 - 拆分 `users / auth_identities / auth_password_credentials` 认证模型，支持注册登录、refresh token 轮换和用户数据隔离；补充关键路径测试与迁移脚本，保证接口与数据库变更可追踪。

- **面试讲解主线：**

“我做这个项目不是为了展示爬虫或 AI，而是为了展示一个真实 Python 后端业务系统怎么从原始数据进入数据库，再到搜索、鉴权、收藏和目标岗位工作台，整个过程中我重点处理了数据分层、幂等去重、查询设计和用户隔离。”

项目二：VoiceMate

- **推荐定位：**

面向绘本陪读场景的 AI 应用后端项目，重点展示多模态上下文组织、会话对象建模、TutorPolicy 策略层、fallback 与实时链路边界。

- **核心亮点 1：围绕陪读场景拆分 backend service 与 ai service**

- 做了什么：把绘本内容访问、会话创建、复盘查询放在 backend service；把上下文组装、模型调用、语音输出、fallback 放在 ai service。
- 为什么这样设计：transport/API 与 AI runtime 的迭代速度、失败模式和部署关注点不同。
- 解决了什么问题：避免一个服务既管资源访问又管实时推理，导致边界混乱。
- 体现什么后端能力：服务边界设计、职责拆分、接口契约。
- 面试时怎么展开讲：从“为什么不做成单体”“两个服务之间交换什么数据”“模型失败时哪一层兜底”展开。

- **核心亮点 2：把自由聊天收敛成可解释的 TutorPolicy 策略层**

- 做了什么：不让模型直接“随便聊”，而是引入 TeachingAction 与 signals，在每个 turn 上做最小教学决策。
- 为什么这样设计：儿童陪读不是开放式聊天，更需要可解释、可复盘、可约束的互动策略。
- 解决了什么问题：降低 AI 输出不可控和“像陪聊机器人”的风险。
- 体现什么后端能力：状态设计、策略层抽象、业务规则落地。
- 面试时怎么展开讲：从“为什么 TutorPolicy 不等于 prompt”“哪些信号由代码控制，哪些由模型决定”展开。

- **核心亮点 3：pipeline-first 的多模态 turn 闭环与 fallback**

- 做了什么：把当前页文本/图像、孩子输入、短会话上下文组装成一个 turn，上下游记录 action / rationale / latency / fallback_used，并输出 review。
- 为什么这样设计：先保证链路可测、可追踪，再做 realtime transport。
- 解决了什么问题：避免项目停留在“接了模型、回了一段话”的玩具 Demo。
- 体现什么后端能力：会话状态管理、可观测性、降级策略、AI 应用工程化。
- 面试时怎么展开讲：从“为什么先 pipeline 后 realtime”“fallback 何时触发”“review 读什么数据生成”展开。

- **可选加分亮点：**

LiveKit token/room info 下发、realtime turn 接通、provider fallback、turn-level telemetry、基础儿童场景安全收敛。

- **不建议强行包装的点：**

通用 Agent 平台、通用 RAG 平台、多 agent 协作、完整教育系统、复杂内容管理后台、本地模型部署矩阵。它们会让项目失焦。

• 简历写法初稿：

- 将绘本陪读场景拆分为 backend service 与 ai service：后端负责绘本内容、会话创建与复盘查询，AI 服务负责上下文组装、策略决策、模型调用与 fallback。
- 设计 `ReadingSession / ReadingTurn / ReadingReview` 运行时对象，并引入 `TutorPolicy` 对每轮交互的教学动作进行显式决策，避免陪读场景退化为自由聊天。
- 采用 pipeline-first 方案验证多模态陪读闭环，记录每轮 latency、fallback 和 review 生成过程，并为 LiveKit / Realtime 预留 token 与房间信息边界。

• 面试讲解主线：

“这个项目的重点不是把大模型接进来，而是把一个具体场景——绘本陪读——拆成 session、turn、policy、response、review 这些清晰对象，让 AI 的能力能被解释、被追踪、被降级。”

你还问了“面试官最可能质疑什么”。下面是我觉得最该正面准备的风险表。

面试风险	更容易出现在	为什么危险	你要准备到什么程度
名词很多，真实链路很少	VoiceMate	一问就暴露是概念项目	现场画出一完整 turn 时序
future 模块过多，已实现模块过少	两个项目都有	像 AI 先搭的空架子	README 明确标注 Implemented / Planned
认证链路讲不清	JobPilot	这是传统后端基本盘	讲清 access/refresh、失效与越权
数据建模讲不清	JobPilot	直接暴露 ORM / SQL 基础不足	能解释为什么拆表、为什么建索引
TutorPolicy 讲不清	VoiceMate	说明你只是“接模型”	能讲 action、signals、fallback
两服务边界讲不清	VoiceMate	显得过度设计	能回答为什么不是单体
性能/延迟数字乱写	两个项目都有	一追问就是虚构	不写没测过的数据
依赖栈堆太多	VoiceMate 尤其明显	像“技术名词壁纸”	删除/降级未使用依赖
监控、MQ、ES 只会说不会用	JobPilot	会被追问得很难看	只保留你真能演示的能力
AI 参与方式说不诚实	两个项目都有	直接影响可信度	明确说“AI 辅助，我负责设计与 review”

JobPilot 可能延伸到的后端八股：

JWT vs Session、refresh token、密码哈希、事务与回滚、唯一约束与并发插入、索引与慢查询、分页、Redis TTL、缓存穿透/击穿、Celery retry、消息 ack、Docker 网络、pytest fixture、配置管理、健康检查。

VoiceMate 可能延伸到的 AI 应用后端问题：

为什么拆 backend/ai service、为什么先 pipeline 后 realtime、如何做 session memory、如何控制 latency、如何收敛 prompt、什么时候触发 fallback、如何记录 turn、如何做 review、为什么不用通用 Agent 框架、如何保证儿童场景基础安全。

如何回答“你在项目中负责什么”：

“我负责需求拆解、接口与对象模型、关键链路设计、AI 生成代码 review、关键模块改写、测试补齐和文档。AI 帮我提升了开发速度，但核心边界和验收标准是我定的。”

如何回答“为什么这么设计”：

“我主要按场景和复杂度做取舍。JobPilot 优先保证数据与查询的真实性，所以先做去重、索引、缓存和用户隔离；VoiceMate 优先保证 AI 链路可解释、可追踪，所以先做 session/turn/policy/fallback，再做 realtime。”

如何回答“如果数据量 / 并发 / 延迟压力变大怎么办”：

- JobPilot：先看查询和索引，再做缓存，再把导入和技能提取异步化，最后才考虑 ES 或读写分离。
- VoiceMate：先缩短上下文、区分快慢路径、把 review 等慢操作异步化、增加 fallback、把 transport 和 policy 分离，再根据需要扩到 realtime service 水平扩容。

一个月执行计划与最终清单

一个月、每天 4 小时左右，我建议你按“先主项目收口，再辅助项目闭环”来排，不要平均用力。Docker Compose 已经很适合你做本地一键启动；pytest fixture 与 `conftest.py` 适合把测试骨架搭好；GitHub Actions 也完全能做最小化 CI；而 VoiceMate 的音频链路根据官方建议先走 pipeline-first 也更符合这个时间预算。¹⁰

周次	每周目标	写代码	review AI 代码	补基础	写文档	面试讲解准备	可验收成果
第 1 周	把 JobPilot 拉到“用户可用闭环”	10h	4h	3h	2h	2h	完成收藏或目标岗位其一；跑通注册登录、岗位搜索、用户动作；补 5 个关键测试；README 增加“已实现/未实现”清单
第 2 周	把 JobPilot 拉到“简历安全线”	10h	4h	3h	2h	2h	完成 <code>user_skills + job match</code> ；把缓存只落在 1-2 个接口；整理 ER 图、导入时序图、接口说明；最小 CI 跑通
第 3 周	把 VoiceMate 做出一个真实 turn 闭环	9h	4h	4h	2h	2h	完成 <code>create session -> pipeline turn -> save turn -> review</code> ；实现最小 TutorPolicy 与 ResponseComposer；补 fallback 与 turn 测试
第 4 周	收口 VoiceMate，并统一做简历/README/讲解材料	8h	4h	3h	4h	3h	完成 VoiceMate 的时序图和非目标说明；如有余力补 room token 接口；整理两个项目的简历描述、90 秒讲解稿和高频问答

最后我把你要的“最终输出”压成一份可以直接执行的清单。

两个项目的定位结论：

JobPilot 是你的主力后端业务系统项目；VoiceMate 是你的AI 应用后端亮点项目。

主项目 / 辅助项目推荐：

主项目：JobPilot。

辅助项目：VoiceMate。

除非你投递的 JD 明确是 AI 应用后端 / 智能体后端，否则不要颠倒顺序。

一个月内最优先做的任务：

先把 JobPilot 做到：认证、岗位导入/去重、岗位查询、用户动作、技能差距分析、关键测试、README 状态清单。

然后把 VoiceMate 做到：session、one-turn pipeline、policy、turn persistence、review、fallback。

必须砍掉的功能：

JobPilot：study_tasks、knowledge、questions、RAG、AI 简历分析、Kafka、复杂 RBAC。

VoiceMate：通用 Agent 平台、通用 RAG 平台、多 agent、复杂后台、本地模型矩阵、完整教育系统。

必须掌握的核心模块：

JobPilot：数据模型、认证链路、导入规范化与 fingerprint 去重、搜索查询与索引、owner-scope 权限、缓存失效、测试。

VoiceMate：backend/ai service 边界、session/turn/review 模型、TutorPolicy、ResponseComposer、fallback、latency budget、room token 责任边界。

适合简历的亮点：

JobPilot 写“导入清洗去重、查询设计、鉴权与用户隔离”。

VoiceMate 写“服务边界、TutorPolicy、pipeline-first turn 闭环、fallback 与 review”。

面试高风险问题：

“为什么这么拆表/拆服务？”

“refresh token 为什么这样做？”

“fingerprint 冲突怎么处理？”

“为什么先 pipeline 后 realtime？”

“TutorPolicy 和 prompt 有什么区别？”

“如果数据量/并发/延迟上来怎么办？”

README 结构建议：

1. 项目简介
2. 问题场景与非目标
3. 当前实现范围
4. 架构图
5. 数据模型 / 运行时对象
6. 关键流程时序图
7. API 清单
8. 技术取舍与为什么不用某些方案
9. 本地启动方式
10. 测试说明
11. 已实现 / 计划中 / 已砍掉
12. 面试可追问题点

简历描述初稿：

• JobPilot

- 设计并实现岗位数据导入与规范化链路，基于 fingerprint 与唯一约束完成幂等去重，并保留 raw/normalized 分层以支持重放与排错。
- 基于 FastAPI + SQLAlchemy 搭建岗位搜索与详情 API，支持多条件筛选、分页与高频结果缓存，围绕查询字段组织数据模型与索引。
- 拆分用户主体、登录身份与密码凭证模型，完成注册登录、refresh token 轮换与用户数据隔离，并补齐关键路径测试与迁移脚本。

• VoiceMate

- 将绘本陪读场景拆分为 backend service 与 ai service，分别负责内容/会话/复盘接口与多模态上下文组装、策略决策、模型调用和 fallback。
- 设计 `ReadingSession / ReadingTurn / ReadingReview` 运行时对象，并通过 `TutorPolicy` 对每轮交互的教学动作进行显式决策。
- 采用 pipeline-first 方案验证多模态陪读闭环，记录 turn latency、fallback 与 review 生成过程，并为 LiveKit / realtime 预留会话与 token 边界。

面试讲解模板：

JobPilot 版本

这个项目我想解决的不是投递管理，而是“岗位情报到求职准备”的后端闭环。我把重点放在三件事：第一是原始岗位数据如何清洗、标准化并幂等去重；第二是围绕搜索筛选去设计数据模型和索引；第三是用户侧的认证、收藏、目标岗位和技能画像如何做隔离。项目里 AI 帮我提高了样板代码开发速度，但数据模型、鉴权链路、去重逻辑、查询条件和测试是我重点 review 和改动的部分。

VoiceMate 版本

这个项目不是做通用 Agent 平台，而是把“绘本陪读”这个具体场景做成一个可解释的 AI 应用后端。我把系统拆成 backend service 和 ai service，前者负责内容、会话、复盘接口，后者负责上下文组装、TutorPolicy、模型调用和 fallback。我的重点不是证明我会接模型，而是证明我能把一次陪读交互拆成 session、turn、policy、response、review 这些稳定对象，并且能解释为什么先做 pipeline、什么时候再上 realtime。

¹ <https://fastapi.tiangolo.com/>

<https://fastapi.tiangolo.com/>

² ⁷ <https://developers.openai.com/api/docs/guides/realtime>

<https://developers.openai.com/api/docs/guides/realtime>

³ ⁶ <https://fastapi.tiangolo.com/features/>

<https://fastapi.tiangolo.com/features/>

⁴ <https://www.postgresql.org/docs/current/textsearch.html>

<https://www.postgresql.org/docs/current/textsearch.html>

⁵ https://docs.sqlalchemy.org/en/latest/changelog/migration_20.html

https://docs.sqlalchemy.org/en/latest/changelog/migration_20.html

⁸ <https://docs.celeryq.dev/en/main/getting-started/backends-and-brokers/>

<https://docs.celeryq.dev/en/main/getting-started/backends-and-brokers/>

⁹ https://prometheus.io/docs/specs/om/open_metrics_spec/

https://prometheus.io/docs/specs/om/open_metrics_spec/

¹⁰ <https://docs.docker.com/compose/>

<https://docs.docker.com/compose/>